

Práctica 09. DML. & CONSULTAS.

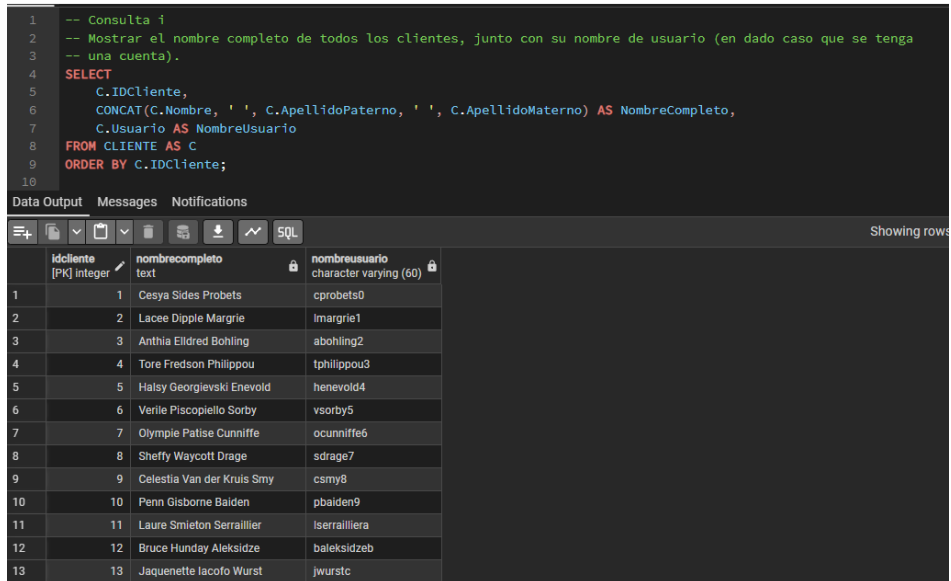
Gallardo Valdez Brayan Alexis
Granados Rovira José Pablo
Lozano León Diego
Márquez Corona Danna Lizette
Paredes González Emiliano Sebastián
Tapia Anrubio Irving Axel

15 Mayo 2026

1. Actividad

Realizar la solución de las siguientes consultas y guardarlas en un archivo llamado `Query.sql`.
Agregar la solución de cada una de las consultas y agregar una screenshot por cada uno de sus resultados:

1. Mostrar el nombre completo de todos los clientes, junto con su nombre de usuario.



```
1  -- Consulta 1
2  -- Mostrar el nombre completo de todos los clientes, junto con su nombre de usuario (en dado caso que se tenga
3  -- una cuenta).
4  SELECT
5      C.IDCliente,
6      CONCAT(C.Nombre, ' ', C.ApellidoPaterno, ' ', C.ApellidoMaterno) AS NombreCompleto,
7      C.Usuario AS NombreUsuario
8  FROM CLIENTE AS C
9  ORDER BY C.IDCliente;
10
```

	idcliente [PK] integer	nombrecompleto text	nombreusuario character varying (60)
1	1	Cesya Sides Proberts	cproberts0
2	2	Lacee Dipple Margrie	lmargrie1
3	3	Anthlia Eldred Bohling	abohling2
4	4	Tore Fredson Philippou	tphilippou3
5	5	Halsy Georgievski Enevold	henevold4
6	6	Verile Piscopiello Sorby	vsorby5
7	7	Olympie Patise Cunniffe	ocunniffe6
8	8	Sheffy Waycott Drage	sdrage7
9	9	Celestia Van der Kruis Smy	csmy8
10	10	Penn Gisborne Baiden	pbaiden9
11	11	Laure Smieton Serrailier	lserrailiera
12	12	Bruce Hunday Aleksidze	baleksidzeb
13	13	Jaquenette Iacofu Wurst	jwurstc

Se utiliza SELECT obtener el identificador del cliente, su nombre completo y el nombre de usuario.

La función CONCAT se utiliza para unir el nombre, apellido paterno y apellido materno del cliente en una sola cadena de texto.

Se utiliza AS para asignar alias a las columnas resultantes.

La cláusula FROM indica que los datos serán obtenidos de la tabla CLIENTE.

La tabla CLIENTE se renombra temporalmente con el alias C.

Finalmente, se utiliza ORDER BY para ordenar los resultados según el identificador de cada cliente.

2. Calcular cuántos medicamentos ha comprado cada cliente.

```
11 -- Consulta 11
12 -- Calcular cuántos medicamentos ha comprado cada cliente.
13 SELECT
14     C.IDCliente,
15     CONCAT(C.Nombre, ' ', C.ApellidoPaterno, ' ', C.ApellidoMaterno) AS NombreCompleto,
16     COUNT(CO.IDMedicamento) AS TotalMedicamentosComprados
17 FROM CLIENTE AS C
18 LEFT JOIN TICKET AS T
19     ON C.IDCliente = T.IDCliente
20 LEFT JOIN COMPRAR AS CO
21     ON T.IDTicket = CO.IDTicket
22 GROUP BY
23     C.IDCliente,
24     C.Nombre,
25     C.ApellidoPaterno,
```

	idcliente [PK] integer	nombrecompleto text	totalmedicamentoscomprados bigint
1	1	Cesya Sides Probets	3
2	3	Anthia Eldred Bohling	3
3	9	Celestia Van der Kruis Smy	3
4	6	Verile Piscopiello Sorby	3
5	2	Lacee Dipple Margrie	3
6	4	Tore Fredson Philippou	3
7	7	Olympie Patise Cuncliffe	3
8	10	Penn Gisborne Baiden	3
9	5	Halsy Georgievski Enevold	3
10	8	Sheffy Waycott Drage	3
11	33	Jennette Isaksen Wince	2
12	24	Jillayne Roubay Niece	2
13	14	Hailee Toms Hanselmann	2

Se utiliza SELECT para obtener el identificador del cliente, su nombre completo y la cantidad de medicamentos comprados.

La función CONCAT se utiliza para unir el nombre y los apellidos del cliente en una sola cadena de texto.

La función COUNT se utiliza para contar la cantidad de medicamentos comprados por cada cliente.

Se utiliza LEFT JOIN para relacionar la tabla CLIENTE con la tabla TICKET.

Posteriormente se utiliza otro LEFT JOIN para relacionar la tabla TICKET con la tabla COMPRAR.

La cláusula ON se utiliza para indicar la condición con la que se relacionan las tablas. La cláusula GROUP BY se utiliza para agrupar los registros por cliente y realizar el conteo correctamente.

Finalmente, se utiliza ORDER BY para ordenar los resultados de mayor a menor según la cantidad de medicamentos comprados.

3. Listar todas las enfermeras cuyo apellido materno contenga "llo".

Query Query History

```

22  GROUP BY
23      C.IDCliente,
24      C.Nombre,
25      C.ApellidoPaterno,
26      C.ApellidoMaterno
27  ORDER BY TotalMedicamentosComprados DESC;
28
29  -- Consulta iii
30  -- Listar todas las enfermeras cuyo apellido materno contenga llo.
31  SELECT
32      P.IDPersonal,
33      CONCAT(P.Nombre, ' ', P.ApellidoPaterno, ' ', P.ApellidoMaterno) AS NombreCompleto,
34      P.ApellidoMaterno,
35      E.CertificadoReanimacion,
36      E.TipoProcedimiento
37  FROM ENFERMERA AS E
38  INNER JOIN PERSONAL AS P
39      ON E.IDPersonal = P.IDPersonal
40  WHERE P.ApellidoMaterno ILIKE '%llo%'
41  ORDER BY P.ApellidoMaterno;
42

```

Data Output Messages Notifications

+

📄

⌵

🗑️

📄

⌵

📄

⌵

📄

⌵

SQL

	idpersonal integer	nombrecompleto text	apellidomaterno character varying (80)	certificadoreanimacion character varying (60)	tipoprocedimiento character varying (100)
1	1001	EnfermeroConsulta1 Prueba Castillo	Castillo	Basico	Apoyo en consulta
2	1002	EnfermeroConsulta2 Prueba Castillo	Castillo	Avanzado	Apoyo en consulta
3	1003	EnfermeroConsulta3 Prueba Castillo	Castillo	Basico	Apoyo en consulta
4	1004	EnfermeroConsulta4 Prueba Castillo	Castillo	Avanzado	Apoyo en consulta
5	1005	EnfermeroConsulta5 Prueba Castillo	Castillo	Basico	Apoyo en consulta
6	1006	EnfermeroConsulta6 Prueba Castillo	Castillo	Avanzado	Apoyo en consulta
7	1007	EnfermeroConsulta7 Prueba Castillo	Castillo	Basico	Apoyo en consulta
8	1008	EnfermeroConsulta8 Prueba Castillo	Castillo	Avanzado	Apoyo en consulta
9	1009	EnfermeroConsulta9 Prueba Castillo	Castillo	Basico	Apoyo en consulta
10	1010	EnfermeroConsulta10 Prueba Cast...	Castillo	Avanzado	Apoyo en consulta

Se utiliza SELECT para obtener el identificador de la enfermera, su nombre completo y datos relacionados con sus certificaciones y procedimientos.

La función CONCAT se utiliza para unir el nombre y los apellidos en una sola cadena de texto.

Se utiliza AS para asignar alias a las columnas y tablas.

La cláusula FROM indica que los datos principales serán obtenidos de la tabla ENFERMERA.

Se utiliza INNER JOIN para relacionar la tabla ENFERMERA con la tabla PERSONAL.

La cláusula ON se utiliza para indicar la condición con la que se relacionan ambas tablas.

La cláusula WHERE se utiliza para filtrar únicamente las enfermeras cuyo apellido materno contenga la cadena llo.

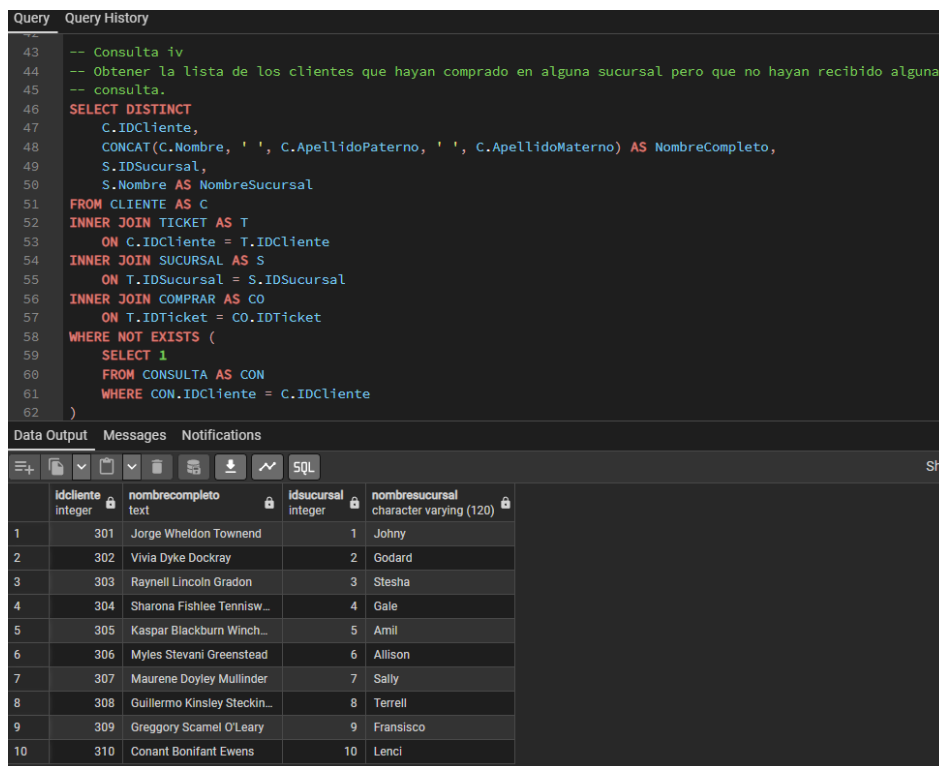
La palabra ILIKE se utiliza para realizar una búsqueda sin distinguir entre mayúsculas y minúsculas.

Los símbolos % se utilizan como comodines para indicar que puede existir cualquier conjunto de caracteres antes o después de la cadena buscada.

Se insertaron registros adicionales de enfermeras con el apellido Castillo para asegurar que la consulta devolviera resultados válidos.

Finalmente, se utiliza ORDER BY para ordenar los resultados según el apellido materno.

4. Obtener la lista de clientes que hayan comprado en alguna sucursal pero que no hayan recibido alguna consulta.



```
43 -- Consulta iv
44 -- Obtener la lista de los clientes que hayan comprado en alguna sucursal pero que no hayan recibido alguna
45 -- consulta.
46 SELECT DISTINCT
47     C.IDcliente,
48     CONCAT(C.Nombre, ' ', C.ApellidoPaterno, ' ', C.ApellidoMaterno) AS NombreCompleto,
49     S.IDsucursal,
50     S.Nombre AS NombreSucursal
51 FROM CLIENTE AS C
52 INNER JOIN TICKET AS T
53     ON C.IDcliente = T.IDcliente
54 INNER JOIN SUCURSAL AS S
55     ON T.IDsucursal = S.IDsucursal
56 INNER JOIN COMPRAR AS CO
57     ON T.IDTicket = CO.IDTicket
58 WHERE NOT EXISTS (
59     SELECT 1
60     FROM CONSULTA AS CON
61     WHERE CON.IDcliente = C.IDcliente
62 )
```

	Idcliente integer	nombrecompleto text	Idsucursal integer	nombresucursal character varying (120)
1	301	Jorge Wheldon Townend	1	Johny
2	302	Vivia Dyke Dockray	2	Godard
3	303	Raynell Lincoln Gradon	3	Stesha
4	304	Sharona Fishlee Tennisw...	4	Gale
5	305	Kaspar Blackburn Winch...	5	Amil
6	306	Myles Stevani Greenstead	6	Allison
7	307	Maurene Doyley Mullinder	7	Sally
8	308	Guillermo Kinsley Steckin...	8	Terrell
9	309	Greggory Scamel O'Leary	9	Fransisco
10	310	Conant Bonifant Ewens	10	Lenci

Se utiliza SELECT para obtener el identificador del cliente, su nombre completo y la sucursal donde realizó compras.

La función CONCAT se utiliza para unir el nombre y los apellidos del cliente en una sola cadena de texto.

Se utiliza DISTINCT para evitar registros duplicados en el resultado.

La cláusula FROM indica que los datos principales serán obtenidos de la tabla CLIENTE.

Se utiliza INNER JOIN para relacionar las tablas CLIENTE, TICKET, SUCURSAL y COMPRAR.

La cláusula ON se utiliza para indicar las condiciones con las que se relacionan las tablas.

La cláusula WHERE se utiliza para aplicar una condición de filtrado.

La cláusula NOT EXISTS se utiliza para verificar que no exista alguna consulta médica asociada al cliente.

La subconsulta se utiliza para buscar registros relacionados dentro de la tabla CONSULTA.

Se insertaron registros adicionales de clientes con compras registradas pero sin consultas médicas para asegurar que la consulta devolviera resultados válidos.

Finalmente, se utiliza ORDER BY para ordenar los resultados según el identificador del cliente.

5. Calcular el precio bruto por ticket.

```

65  -- Consulta v
66  -- Calcular el precio bruto por ticket.
67  |
68  SELECT
69      T.IDTicket,
70      T.IDCliente,
71      SUM(CO.Cantidad * M.PrecioPublico) AS PrecioBruto
72  FROM TICKET AS T
73  INNER JOIN COMPRAR AS CO
74      ON T.IDTicket = CO.IDTicket
75  INNER JOIN MEDICAMENTO AS M
76      ON CO.IDMedicamento = M.IDMedicamento
77  GROUP BY
78      T.IDTicket,
79      T.IDCliente
80  ORDER BY T.IDTicket;
81

```

Data Output Messages Notifications

	Idticket [PK] integer	Idcliente integer	preciobruto numeric
1	1	1	526.64
2	2	2	1540.18
3	3	3	1290.36
4	4	4	2966.48
5	5	5	509.40
6	6	6	736.45
7	7	7	505.88
8	8	8	307.17
9	9	9	822.92
10	10	10	875.95
11	11	11	179.05
12	12	12	1035.16
13	13	13	1359.06

Se utiliza SELECT para obtener el identificador del ticket, el identificador del cliente y el precio bruto calculado.

La función SUM se utiliza para sumar el costo total de los medicamentos asociados a cada ticket.

La multiplicación se utiliza para calcular el costo total de cada medicamento considerando la cantidad comprada y el precio público.

Se utiliza AS para asignar alias a las columnas y tablas.

La cláusula FROM indica que los datos principales serán obtenidos de la tabla TICKET. Se utiliza INNER JOIN para relacionar la tabla TICKET con las tablas COMPRAR y MEDICAMENTO.

La cláusula ON se utiliza para indicar las condiciones con las que se relacionan las tablas.

La cláusula GROUP BY se utiliza para agrupar los registros por ticket y cliente y realizar correctamente la suma de los precios.

Finalmente, se utiliza ORDER BY para ordenar los resultados según el identificador del ticket.

6. Calcular el precio neto por ticket.

Query

Query History

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

-- Consulta v1
-- Calcular el precio neto por ticket.
-- Se considera IVA del 16% sobre el precio bruto.

SELECT
 T.IDTicket,
 T.IDCliente,
 SUM(CO.Cantidad * M.PrecioPublico) AS PrecioBruto,
 ROUND((SUM(CO.Cantidad * M.PrecioPublico) * 1.16)::numeric, 2) AS PrecioNeto
FROM TICKET AS T
INNER JOIN COMPRAR AS CO
 ON T.IDTicket = CO.IDTicket
INNER JOIN MEDICAMENTO AS M
 ON CO.IDMedicamento = M.IDMedicamento
GROUP BY
 T.IDTicket,
 T.IDCliente
ORDER BY T.IDTicket;

Data Output

Messages

Notifications

SQL

	Idticket [PK] integer	Idcliente integer	preciobruto numeric	precioneto numeric
1	1	1	526.64	610.90
2	2	2	1540.18	1786.61
3	3	3	1290.36	1496.82
4	4	4	2966.48	3441.12
5	5	5	509.40	590.90
6	6	6	736.45	854.28
7	7	7	505.88	586.82
8	8	8	307.17	356.32
9	9	9	822.92	954.59
10	10	10	875.95	1016.10
11	11	11	179.05	207.70
12	12	12	1035.16	1200.79
13	13	13	1359.06	1576.51

Se utiliza SELECT para obtener el identificador del ticket, el identificador del cliente, el precio bruto y el precio neto.

La función SUM se utiliza para sumar el costo total de los medicamentos asociados a cada ticket.

La multiplicación se utiliza para calcular el costo total considerando la cantidad comprada y el precio público del medicamento.

Se utiliza ROUND para redondear el precio neto a dos decimales.

La expresión $\times 1.16$ se utiliza para agregar el IVA del 16 % al precio bruto.

La conversión numeric se utiliza para convertir el resultado a tipo numérico y permitir el redondeo correcto.

Se utiliza AS para asignar alias a las columnas y tablas.

La cláusula FROM indica que los datos principales serán obtenidos de la tabla TICKET.

Se utiliza INNER JOIN para relacionar la tabla TICKET con las tablas COMPRAR y MEDICAMENTO.

La cláusula ON se utiliza para indicar las condiciones con las que se relacionan las tablas.

La cláusula GROUP BY se utiliza para agrupar los registros por ticket y cliente y realizar correctamente la suma de los precios.

Finalmente, se utiliza ORDER BY para ordenar los resultados según el identificador del ticket.

7. Calcular el precio total pagado por cada cliente.

Query

Query History

```
103 -- Consulta vii
104 -- Calcular el precio total que ha pagado cada cliente.
105 -- Se suma el precio neto de todos sus tickets.
106
107 WITH PrecioTicket AS (
108     SELECT
109         T.IDTicket,
110         T.IDCliente,
111         ROUND((SUM(CO.Cantidad * M.PrecioPublico) * 1.16)::numeric, 2) AS PrecioNeto
112     FROM TICKET AS T
113     INNER JOIN COMPRAR AS CO
114         ON T.IDTicket = CO.IDTicket
115     INNER JOIN MEDICAMENTO AS M
116         ON CO.IDMedicamento = M.IDMedicamento
117     GROUP BY
118         T.IDTicket,
119         T.IDCliente
120 )
121 SELECT
122     C.IDCliente,
```

Data Output

Messages

Notifications

SQL

	Idcliente [PK] integer	nombrecompleto text	totalpagado numeric
1	25	Obadiah Shelsher Espinel	5508.00
2	15	Dorthy Cardero McMaster	5450.95
3	40	Kylen Doddemeade Yuryatin	5075.84
4	4	Tore Fredson Philippou	4964.24
5	260	Thedrick Romushkin Goodere	4528.23
6	90	Nicky Kneath Whitchurch	4433.98
7	70	Kippy Mandifield Curnucke	4397.33
8	95	Edita Dunan Cudiff	4375.93
9	135	Rivkah Mieville Duxbarry	4374.19
10	235	Stacy Dakin Spadoni	4258.53
11	185	Elizabeth Goulder Dorbin	3978.80
12	125	Tanny Santori Toy	3830.09
13	35	Elfreda Kubat Blankau	3818.19

La cláusula WITH se utiliza para crear una consulta temporal llamada PrecioTicket.

La subconsulta PrecioTicket se utiliza para calcular el precio neto de cada ticket.

Se utiliza SELECT para obtener el identificador del cliente, su nombre completo y el total pagado.

La función CONCAT se utiliza para unir el nombre y los apellidos del cliente en una sola cadena de texto.

La función SUM se utiliza para sumar el precio neto de todos los tickets asociados a cada cliente.

La función COALESCE se utiliza para reemplazar valores nulos por cero en caso de que un cliente no tenga compras registradas.

Se utiliza LEFT JOIN para relacionar la tabla CLIENTE con la consulta temporal PrecioTicket.

La cláusula ON se utiliza para indicar la condición con la que se relacionan los registros.

La cláusula GROUP BY se utiliza para agrupar los registros por cliente y realizar correctamente la suma total pagada.

Finalmente, se utiliza ORDER BY para ordenar los resultados de mayor a menor según el total pagado.

8. Listar a los enfermeros que atendieron alguna consulta durante el 7 de mayo del 2026 entre las 12:00 hrs y las 16:00 hrs.

Query		Query History	
136	-- Consulta viii		
137	-- Listar a los enfermeros que atendieron alguna consulta durante el		
138	-- 7 de mayo de 2026, de 12:00 hrs a 16:00 hrs.		
139			
140	SELECT DISTINCT		
141	P.IDPersonal,		
142	CONCAT(P.Nombre, ' ', P.ApellidoPaterno, ' ', P.ApellidoMaterno) AS NombreCompleto,		
143	E.CertificadoReanimacion,		
144	E.TipoProcedimiento,		
145	CON.IDConsulta,		
146	CON.Fecha,		
147	CON.Hora		
148	FROM CONSULTA AS CON		
149	INNER JOIN ENFERMERA AS E		
150	ON CON.IDEnfermera = E.IDPersonal		
151	INNER JOIN PERSONAL AS P		
152	ON E.IDPersonal = P.IDPersonal		
153	WHERE CON.Fecha = DATE '2026-05-07'		
154	AND CON.Hora BETWEEN TIME '12:00' AND TIME '16:00'		
155	ORDER BY		
Data Output			
Messages			
Notifications			
SQL			
	idpersonal	nombrecompleto	certificadoreanimacion
	integer	text	character varying (50)
			tipoprocedimiento
			character varying (100)
			idconsulta
			integer
			fecha
			date
			hora
			time without time zone
1	1001	EnfermeroConsulta1 Prueba Castillo	Basico
2	1006	EnfermeroConsulta6 Prueba Castillo	Avanzado
3	1002	EnfermeroConsulta2 Prueba Castillo	Avanzado
4	1007	EnfermeroConsulta7 Prueba Castillo	Basico
5	1003	EnfermeroConsulta3 Prueba Castillo	Basico
6	1008	EnfermeroConsulta8 Prueba Castillo	Avanzado
7	1004	EnfermeroConsulta4 Prueba Castillo	Avanzado
8	1009	EnfermeroConsulta9 Prueba Castillo	Basico
9	1005	EnfermeroConsulta5 Prueba Castillo	Basico
10	1010	EnfermeroConsulta10 Prueba Cast...	Avanzado

Se utiliza SELECT para obtener la información de las enfermeras y las consultas atendidas.

La función CONCAT se utiliza para unir el nombre y los apellidos de la enfermera en una sola cadena de texto.

Se utiliza DISTINCT para evitar registros duplicados en el resultado.

La cláusula FROM indica que los datos principales serán obtenidos de la tabla CONSULTA.

Se utiliza INNER JOIN para relacionar las tablas CONSULTA, ENFERMERA y PERSONAL.

La cláusula ON se utiliza para indicar las condiciones con las que se relacionan las tablas.

La cláusula WHERE se utiliza para filtrar únicamente las consultas realizadas el 7 de mayo de 2026.

La cláusula WITH se utiliza para crear una consulta temporal llamada ProductosProveedor.

Se utiliza SELECT para obtener el identificador del proveedor, el tipo de producto, el producto y el precio unitario.

La palabra 'Medicamento' se utiliza para identificar los registros correspondientes a medicamentos.

La conversión ::text se utiliza para convertir el identificador del medicamento a tipo texto y mantener compatibilidad con la operación de unión.

Se utiliza INNER JOIN para relacionar las tablas PROVEERMEDICAMENTO y MEDICAMENTO.

Posteriormente se utiliza otro SELECT para obtener los insumos proporcionados por cada proveedor.

La palabra 'Insumo' se utiliza para identificar los registros correspondientes a insumos.

Se utiliza INNER JOIN para relacionar las tablas PROVEERINSUMO e INSUMO.

La cláusula UNION ALL se utiliza para combinar los resultados de medicamentos e insumos en una sola tabla temporal sin eliminar registros repetidos.

En la consulta principal se utiliza LEFT JOIN para relacionar la tabla PROVEEDOR con la consulta temporal ProductosProveedor.

La cláusula ON se utiliza para indicar la condición con la que se relacionan los registros.

Finalmente, se utiliza ORDER BY para ordenar los resultados por proveedor, tipo de producto y producto.

10. Mostrar las sucursales que posean al menos 5 médicos.

Query

Query History

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

```

-- Consulta x
-- Mostrar las sucursales que posean al menos 5 medicos.

SELECT
    S.IDSucursal,
    S.Nombre AS NombreSucursal,
    COUNT(ME.IDPersonal) AS TotalMedicos
FROM SUCURSAL AS S
INNER JOIN PERSONAL AS P
    ON S.IDSucursal = P.IDSucursal
INNER JOIN MEDICO AS ME
    ON P.IDPersonal = ME.IDPersonal
GROUP BY
    S.IDSucursal,
    S.Nombre
HAVING COUNT(ME.IDPersonal) >= 5
ORDER BY TotalMedicos DESC;

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🔄

⬇️

📈

SQL

	idsucursal [PK] integer	nombresucursal character varying (120)	totalmedicos bigint
1	8	Terrell	7
2	6	Allison	7
3	7	Sally	6
4	1	Johny	6
5	10	Lenci	6
6	2	Godard	5
7	5	Amil	5
8	3	Stesha	5
9	4	Gale	5
10	9	Fransisco	5

Se utiliza SELECT para obtener el identificador de la sucursal, el nombre de la sucursal y la cantidad total de médicos.

La función COUNT se utiliza para contar la cantidad de médicos asociados a cada sucursal.

Se utiliza AS para asignar alias a las columnas y tablas.

La cláusula FROM indica que los datos principales serán obtenidos de la tabla SUCURSAL.

Se utiliza INNER JOIN para relacionar las tablas SUCURSAL, PERSONAL y MEDICO.

La cláusula ON se utiliza para indicar las condiciones con las que se relacionan las tablas.

La cláusula GROUP BY se utiliza para agrupar los registros por sucursal y realizar correctamente el conteo de médicos.

La cláusula HAVING se utiliza para filtrar únicamente las sucursales que tengan al menos 5 médicos.

Se insertaron registros adicionales de médicos distribuidos entre distintas sucursales para asegurar que algunas cumplieran con la condición mínima de 5 médicos.

Finalmente, se utiliza ORDER BY para ordenar los resultados de mayor a menor según la cantidad de médicos.

11. Listar a los vendedores cuyo total de alimentos vendidos sea mayor a 3.

```
219 --- Consulta xi
220 -- Listar a los vendedores cuyo total de productos distintos vendidos sea mayor a 3.
221 -- En este modelo se interpreta "vendedores" como sucursales.
222
223 SELECT
224     S.IDSucursal,
225     S.Nombre AS NombreSucursal,
226     COUNT(DISTINCT CO.IDMedicamento) AS ProductosDistintosVendidos
227 FROM SUCURSAL AS S
228 INNER JOIN TICKET AS T
229     ON S.IDSucursal = T.IDSucursal
230 INNER JOIN COMPRAR AS CO
231     ON T.IDTicket = CO.IDTicket
232 GROUP BY
233     S.IDSucursal,
234     S.Nombre
235 HAVING COUNT(DISTINCT CO.IDMedicamento) > 3
236 ORDER BY ProductosDistintosVendidos DESC;
```

	idsucursal [PK] integer	nombresucursal character varying (120)	productosdistintosvendidos bigint
1	6	Allison	8
2	9	Fransisco	8
3	8	Terrell	8
4	5	Amil	7
5	7	Sally	7
6	10	Lenci	7
7	2	Godard	7
8	4	Gale	7
9	1	Johnny	7
10	3	Stesha	6

En el documento se indicaba alimentos, sin embargo nosotros colocamos Medicamentos.

Se utiliza SELECT para obtener el identificador de la sucursal, el nombre de la sucursal y la cantidad de productos distintos vendidos.

La función COUNT se utiliza para contar los productos vendidos.

La palabra DISTINCT se utiliza para contar únicamente productos diferentes y evitar duplicados.

Se utiliza AS para asignar alias a las columnas y tablas.

La cláusula FROM indica que los datos principales serán obtenidos de la tabla SUCURSAL.

Se utiliza INNER JOIN para relacionar las tablas SUCURSAL, TICKET y COMPRAR.

La cláusula ON se utiliza para indicar las condiciones con las que se relacionan las tablas.

La cláusula GROUP BY se utiliza para agrupar los registros por sucursal y realizar correctamente el conteo de productos distintos vendidos.

La cláusula HAVING se utiliza para filtrar únicamente las sucursales cuyo total de productos distintos vendidos sea mayor a 3.

Se insertaron registros adicionales de tickets y compras para asegurar que algunas sucursales tuvieran más de 3 medicamentos distintos vendidos.

Finalmente, se utiliza ORDER BY para ordenar los resultados de mayor a menor según la cantidad de productos distintos vendidos.

12. Listar a los proveedores cuyo total de productos que proveen sea mayor a 3.

Query

Query History

239

-- Consulta xii

240

-- Listar a los proveedores cuyo total de productos distintos que proveen

241

-- sea mayor a 3.

242

243

WITH ProductosProveedor AS (

244

SELECT

245

IDProveedor,

246

'MED-' || IDMedicamento::text AS Producto

247

FROM PROVEER_MEDICAMENTO

248

249

UNION

250

251

SELECT

252

IDProveedor,

253

'INS-' || NombreCientifico AS Producto

254

FROM PROVEER_INSUMO

255

)

256

SELECT

257

PR.IDProveedor,

258

PR.RazonSocial,

Data Output

Messages

Notifications

SQL

	idproveedor [PK] integer	razonsocial character varying (150)	productosdistintosprovistos bigint
1	1	Demivee	4
2	2	Zazio	4
3	3	Rhyzio	4
4	4	Digitube	4
5	5	Chatterpoint	4
6	6	Muxo	4
7	7	Gabtype	4
8	8	Rooxo	4
9	9	Yambee	4
10	10	Zava	4
11	11	Zoomzone	4
12	12	Gabvine	4
13	13	Kwideo	4

La cláusula WITH se utiliza para crear una consulta temporal llamada ProductosProveedor.

Se utiliza SELECT para obtener el identificador del proveedor y el producto proporcionado.

La concatenación || se utiliza para unir texto y generar un identificador único para cada producto.

La conversión ::text se utiliza para convertir el identificador del medicamento a tipo texto.

La palabra 'MED-' se utiliza para identificar productos correspondientes a medicamentos.

La palabra 'INS-' se utiliza para identificar productos correspondientes a insumos.

La cláusula UNION se utiliza para combinar los productos provenientes de medicamentos e insumos eliminando duplicados.

En la consulta principal se utiliza COUNT para contar la cantidad de productos distintos proporcionados por cada proveedor.

La palabra DISTINCT se utiliza para contar únicamente productos diferentes.

Se utiliza INNER JOIN para relacionar la tabla PROVEEDOR con la consulta temporal ProductosProveedor.

La cláusula ON se utiliza para indicar la condición con la que se relacionan los registros.

La cláusula GROUP BY se utiliza para agrupar los registros por proveedor y realizar correctamente el conteo.

La cláusula HAVING se utiliza para filtrar únicamente los proveedores que proporcionan más de 3 productos distintos.

Finalmente, se utiliza ORDER BY para ordenar los resultados de mayor a menor según la cantidad de productos distintos provistos.

13. Obtener las ganancias y pérdidas totales por cada sucursal.

Query

Query History

```
270 -- Consulta xiii
271 -- Obtener ganancias y perdidas totales por cada sucursal.
272 -- Ganancia: venta de medicamentos por ticket.
273 -- Perdida: costo de productos suministrados por proveedores.
274
275 WITH Ganancias AS (
276     SELECT
277         T.IDSucursal,
278         SUM(CO.Cantidad * M.PrecioPublico) AS GananciaBruta
279     FROM TICKET AS T
280     INNER JOIN COMPRAR AS CO
281         ON T.IDTicket = CO.IDTicket
282     INNER JOIN MEDICAMENTO AS M
283         ON CO.IDMedicamento = M.IDMedicamento
284     GROUP BY T.IDSucursal
285 ),
286 PerdidasMedicamento AS (
287     SELECT
288         PM.IDSucursal,
289         SUM(PM.Cantidad * M.PrecioUnitario) AS PerdidaMedicamentos
290     FROM PROVEEDOR_MEDICAMENTO AS PM
```

Data Output

Messages

Notifications

SQL

	idsucursal [PK] integer	nombresucursal character varying (120)	gananciastotales numeric	perdidastotales numeric	utilidadneta numeric
1	19	Hunt	1242.32	22370.36	-21128.04
2	26	Micheil	875.25	26934.83	-26059.58
3	177	Lynn	633.16	29008.00	-28374.84
4	75	Polly	1849.05	61168.58	-59319.53
5	104	Alvira	1141.62	67541.27	-66399.65
6	178	Davide	2296.36	69130.25	-66833.89
7	6	Allison	8220.15	83774.11	-75553.96
8	99	Philippe	2085.42	86988.54	-84903.12
9	70	Sigfried	2840.80	89801.21	-86960.41
10	143	Dietrich	2291.36	89816.80	-87525.44
11	59	Temp	1540.06	89828.54	-88288.48
12	180	Skelly	3086.40	91845.59	-88759.19
13	87	Dorian	179.61	89714.10	-89534.49

La cláusula WITH se utiliza para crear consultas temporales llamadas Ganancias, PerdidasMedicamento y PerdidasInsumo.

La consulta temporal Ganancias se utiliza para calcular las ganancias obtenidas por cada sucursal mediante la venta de medicamentos.

La función SUM se utiliza para sumar los ingresos y pérdidas correspondientes a cada sucursal.

La multiplicación se utiliza para calcular el costo total considerando la cantidad y el precio unitario o precio público.

Se utiliza INNER JOIN para relacionar las tablas involucradas en el cálculo de ganancias y pérdidas.

La cláusula ON se utiliza para indicar las condiciones con las que se relacionan las tablas.

La consulta temporal PerdidasMedicamento se utiliza para calcular el costo de los medicamentos suministrados por proveedores.

La consulta temporal PerdidasInsumo se utiliza para calcular el costo de los insumos suministrados por proveedores.

En la consulta principal se utiliza SELECT para obtener el identificador de la sucursal, las ganancias totales, las pérdidas totales y la utilidad neta.

La función COALESCE se utiliza para reemplazar valores nulos por cero en caso de que una sucursal no tenga ganancias o pérdidas registradas.

Se utiliza LEFT JOIN para relacionar la tabla SUCURSAL con las consultas temporales y asegurar que todas las sucursales aparezcan en el resultado.

La suma se utiliza para calcular las pérdidas totales considerando medicamentos e insumos.

La resta se utiliza para calcular la utilidad neta restando las pérdidas a las ganancias.

Finalmente, se utiliza ORDER BY para ordenar los resultados de mayor a menor según la utilidad neta.

Nota: Se hicieron modificaciones a los diagramas que hacian falta y consecuentemente a todo lo demas